Rethinking Consensus with Time as a Primitive

Ignacio Amores-Sesar[1] and Michelle Yeo[2]

[1]Aarhus University, Denmark
[2]National University of Singapore, Singapore

Abstract

We propose a novel timestamping mechanism for consensus protocols that reliably assigns submission times to honest transactions while preventing adversarial transactions from forging their timestamps. Our mechanism remains secure even under asynchronous networks and in the presence of corrupted parties. We demonstrate how it can be integrated into the three main families of consensus protocols and show its compatibility with layer-2 solutions. This construction enables robust layer-2 implementations that do not rely on timing assumptions, remain resilient under network saturation, and prevent collusion between users and validators to censor honest users.

1 Introduction

Consensus algorithms form the foundation for secure and decentralized online payment systems. By allowing a distributed set of nodes to agree on the sequence and validity of operations, these protocols replace the need for centralized financial authorities in traditional finance. In this context, blockchain protocols offer a compelling alternative to traditional payment mechanisms. However, their fundamental reliance on consensus over all the nodes in the system for each operation imposes a significant scalability bottleneck: every transaction must be broadcast and confirmed by the entire network, resulting in limited throughput and latency challenges. Efforts to address these performance constraints have followed two main directions: (1) improving the efficiency of consensus protocols themselves, and (2) offloading work from the consensus layer by enabling off-chain execution, where consensus is used only to resolve disputes or finalize state transitions.

To address direction (1), researchers have proposed a range of consensus algorithms that aim to improve performance directly. In particular, DAG-based consensus protocols [29, 18, 44, 30, 24, 42, 14, 9, 8] have emerged as a promising direction. By enabling blocks to be produced and validated in parallel, DAG protocols achieve significantly higher throughput than traditional chain-based systems. However, these gains in performance come at the cost of transaction order order: the total order of DAG-based protocols is not predictable before the execution of transactions takes place. As a result, the dispute-resolution mechanisms used in off-chain solutions built on longest-chain protocols (like Bitcoin or Ethereum) cannot be easily ported to DAG-based systems.

To address direction (2), off-chain solutions are increasingly proposed as alternatives to full consensus over the whole network and consequently are promising solutions to reduce the bloat of the consensus protocol. Some notable examples of off-chain payment mechanisms are rollups [1, 2, 4, 5, 33] and payment channel networks [11, 19, 20, 38, 40, 45]. These solutions offer scalability by reducing the frequency with which the consensus layer is engaged. For instance, a bi-directional payment channel can facilitate an unbounded number of transactions over extended periods without ever interacting with the underlying blockchain, as long as the balance constraints of the users are respected. These so-called layer 2 solutions, however, come with critical limitations. Their security is tightly coupled to timing assumptions: users must be able to contest fraudulent state updates within narrowly defined windows. This requirement introduces liveness dependencies on user behavior and reliable network access. In practice, these timing assumptions become fragile in the presence of network congestion or high transaction

fees. During periods of saturation, users may be priced out of the opportunity to dispute invalid state transitions, effectively weakening the security guarantees of the system. Moreover, high-value layer 2 transactions are vulnerable to censorship attacks—where honest user transactions are excluded from the chain—or to bribery attacks, where adversaries incentivize miners to ignore dispute messages [36, 12].

In this work, we introduce a timestamping mechanism that compensates for the lack of stability and fairness of some consensus protocols and of more traditional consensus protocols during periods of high-saturation.

Our contribution. The aim of our work is to bridge the gap between the aforementioned two directions of research. We propose a timestamping mechanism that equips consensus protocols with a notion of submission time—allowing transactions to be associated with a globally recognized temporal order, even in asynchronous or partially synchronous settings. Our approach leverages local timestamps reported by participating nodes, while incorporating robust filtering and aggregation techniques to mitigate adversarial bias from Byzantine actors.

The resulting timestamp function provides sufficient temporal structure to support the implementation of layer 2 protocols, even in systems where traditional block ordering is unavailable or unreliable. In particular, we show how timestamping can relax the timing assumptions of existing off-chain protocols, enabling dispute resolution to occur without strong synchrony guarantees. The only requirement is that communication between users and the consensus protocol is eventually synchronous.

By decoupling the reliance on order predictability and mitigating timing-based attacks, our timestamping abstraction opens the door to efficient, secure, and flexible payment protocols on top of modern high-throughput consensus systems.

Organization. Section 1.1 provides an overview of research related to our work. In Section 2, we present our system and adversarial model, as well as the desired properties we want our timestamping mechanism to satisfy. We then go on to describe how to concretely implement our timestamping mechanism for canonical examples of DAG-based, leader-based, and longest chain protocols in Section 3, and prove that these implementations satisfy our desired properties as specified in Section 2. In Section 4, we define ideal interactions between our consensus and layer 2 protocols and show that our augmenting consensus protocols with our timestamping mechanism allows us to build secure layer 2 protocols on top of our consensus protocols. We conclude our work in Section 5.

1.1 Related Work

Timing assumptions for blockchains have been introduced in the initial work of Tzinas et al. [48]. In particular, this work studies the question of how closely on-chain timestamps correspond to real-world time and answers the question in the affirmative for proof-of-work (PoW) and proof-of-stake (both longest chain and quorum-based) blockchain protocols. Our work continues this line of analysis and expands upon this in three distinct ways. First, our protocol extends the notion of timestamping to not just include blocks but also individual transactions and parties, and guarantees censorship resistance for honest parties, i.e., that all honest transactions submitted before a certain time will be delivered. Second, we show that our proposed time-stamping protocol also works on DAG-based protocols [9, 14, 18, 29, 43, 44], which are notoriously difficult to analyze and were not included in the analysis of [48]. Finally, we note that the analysis of [48] does not consider security of layer 2 solutions against various attacks that exploit transaction censorship (e.g., bribing attacks [12]). In contrast, the censorship resistance guarantees in our work preclude such attacks on layer 2.

Our proposed timestamping protocol enables generic layer 2 solutions to be built on top of any family of consensus protocols, and additionally ensures safety in periods of network saturation. Examples of such layer 2 solutions are payment channel networks [11, 19, 20, 38, 40, 45], rollups [1, 2, 4, 5, 33], sidechains [3, 6, 27, 31], and hybrid layer 2 solutions [15, 28, 47, 50]. These layer 2 solutions usually rely on a "contention-period" where parties have some number of blocks to contest fraudulent layer

2 states submitted to the blockchain by submitting an alternative, efficiently-verifiable state (see [22] for a comprehensive survey). The timestamping protocol proposed in our work guarantees censorship resistance for honest parties, which enables honest parties to securely contest fraudulent states during the contention period present in these layer 2 solutions.

Our work is also motivated by time-based attacks on blockchains and layer 2, like bribing [10, 12, 13], and feather forking attacks [35, 37, 41]. These attacks exploit either the timing of the layer 2 contention period or the confirmation period of the underlying blockchain to exclude certain transactions from the blockchain. More generally, our work addresses the issue of blockchain censorship [49, 51], in particular transaction-level censorship attacks, which exclude certain transactions for the purpose of increasing maximal extractable value [49].

Further upfield but also related are preliminary layer 2 solutions for DAG-based blockchain protocols. Thus far, such solutions are limited to "fast-path" protocols like that proposed in Sui [46]. These fast-path protocols can immediately process and execute transactions with consensus number of 1 [25, 26], that is, transactions without any dependencies (e.g., transactions without concurrent data access) or immutable transactions, bypassing the common consensus process for such transactions. Such limited use cases, however, preclude canonical layer 2 solutions like payment channels which involve updating states of objects that are owned by multiple users. In contrast, the timestamping protocol proposed in our work introduces timing guarantees of the form of delivery of honest transactions broadcasted before a certain timestamp for DAG-based blockchains, which allows generic layer 2 solutions to be implemented over DAG-based blockchains.

2 Model

Parties and adversary. We assume a set of n parties $\mathcal{N} = \{1, 2, ..., n\}$, up to f of them can be corrupted by the adversary. Corrupted parties, also referred to as *Byzantine parties*, follow the instructions of the adversary and may deviate arbitrarily from the execution of the protocols. We assume the rest of the parties are honest and follow the stipulated protocols. Additionally, we assume a set of users that create and submit transactions to the protocols, following the submission instructions of the corresponding protocol. The set of users is independent from the set of parties, and the adversary has no corruption threshold regarding users. The set of parties may be known or unknown, depending on the protocol studied.

Communication. We say a network is *synchronous* if all messages sent at time t are delivered by time $t + \Delta$ for some globally known $\Delta > 0$. A network is *asynchronous* if messages can be delayed by adversarial parties for an arbitrary but finite amount of time. The messages, however, need to be eventually delivered. In both cases the adversary determines the exact time of delivery of the messages.

We assume a dual communication model. On the one hand, communication between parties may be synchronous with known network delay Δ_v or asynchronous, depending on the underlying consensus protocol. On the other hand, the communication between users and parties is assumed to be synchronous. This means that a message sent from a user to a party is guaranteed to be delivered within time $\Delta_u > 0$. We also assume that every honest party and user has a local clock indicating time and the clock may drift from the real time by at most Δ_c.

Apart from asynchronous protocols, this model captures the situation in which synchronous protocols are saturated. For instance, consider a protocol like Ethereum [21] during a time in which new coins of its ecosystem have been launched. In order to get a transaction executed within a particular time limit, users may need to pay a fee orders of magnitude above the usual value. Consider a user of a payment channel who needs to post the most recent state to prevent a malicious counterpart from stealing the locked funds. Such a party may not be able to post the transaction in time, without paying a fee equivalent to or higher than the locked funds.

Timestamp functionality. The realm of consensus protocols is rich in diversity. Ranging from the traditional leader-based protocols such as Paxos [34] or PBFT [17] to the latest DAG-based algorithms [29, 18, 44, 30, 24, 42, 14, 9, 8] and longest-chain protocols like Bitcoin [39] and Ethereum [21]. Thus, we describe a set of general properties that allow fair and secure timestamping of consensus protocols to explain later how to implement such properties in the aforementioned three main families of consensus protocols in the corresponding sections.

In a nutshell, our timestamping functionality $\tau(\cdot)$ is accessed with different arguments: a block B, a transaction T, a pair (i, t) formed by party i and time t, or a state of the protocol $\mathit{VIEW}(i, t)$, the view of some party i at time t. For a given block B, $\tau(B)$ returns the *block timestamp* of B. If the argument to τ is a transaction T, we use the term *transaction timestamp* to denote the output of $\tau(T)$. When given a pair (i, t), the output of $\tau(i, t)$ is the *local timestamp* of party i at time t. When given $\mathit{VIEW}(i, t)$, the view of party i at time i, $\tau(\mathit{VIEW}(i, t))$ returns the *global timestamp* of party i at time t.

Intuitively, the local timestamp of a party serves as an upper bound which guarantees that any transaction received by some time determined by a function of the local timestamp will always be delivered by the party. The global timestamp is a timestamp that is computed from sufficiently many local timestamps of honest parties such that it is known and adopted by all honest parties, as well as provides a guarantee for all honest parties that any transaction submitted before the global timestamp will be delivered by all honest parties. We also introduce the protocol assigned time for a block B or a transaction T. Intuitively, $\mathit{time}(T)$ reflects the time t such that at least an honest party knows about the existence of transaction T, similar for a block B. The precise details of the timestamping functionality (including the details when called with different arguments) varies across the different consensus protocols. We introduce the precise timestamping functionality for three representatives of the main families of consensus protocols: *leader-based* (PBFT [17]), *DAG-Based* (DAG-Rider [29]), and *longest-chain* (Nakamoto [39]).

It is paramount that introducing timestamping does not interfere with the safety and liveness properties of atomic broadcast [16]. Our atomic broadcast primitive is accessed through multiple events $\mathit{ab\text{-}broadcast}(B, i, r)$ and outputs events $\mathit{ab\text{-}deliver}(B, j, r)$, where B denotes the block (i.e., message) and r denotes the round. The event $\mathit{ab\text{-}broadcast}(B, i, r)$ can only be triggered by party i, however, the events $\mathit{ab\text{-}deliver}(B, j, r)$ may be output by every party.

Definition 1 (Atomic broadcast). A protocol solves *atomic broadcast* if it satisfies the following properties, except with negligible probability:

Validity. If an honest party i invokes $\mathit{ab\text{-}broadcast}(B, i, r)$ after the global stabilization time, then party i eventually outputs $\mathit{ab\text{-}deliver}(B, i, r)$.

Agreement. If an honest party outputs $\mathit{ab\text{-}deliver}(B, i, r)$, then all honest parties eventually output $\mathit{ab\text{-}deliver}(B, i, r)$.

Integrity. For any party i and round r, every honest party outputs the event $\mathit{ab\text{-}deliver}(B, i, r)$ at most once regardless of B. Moreover, if i is honest, then i has previously invoked $\mathit{ab\text{-}broadcast}(B, i, r)$.

Total order. If an honest party outputs the event $\mathit{ab\text{-}deliver}(B, i, r)$ before the event $\mathit{ab\text{-}deliver}(B', j, r')$, then no honest party outputs $\mathit{ab\text{-}deliver}(B', j, r')$ before $\mathit{ab\text{-}deliver}(B, i, r)$.

In the following, we define our main desired property, *timestamped atomic broadcast*, which is an enhanced variant of atomic broadcast that includes the concept of time of submission (timestamp) and execution (time) of transactions.

Definition 2 (Timestamped atomic broadcast). Given an atomic broadcast protocol Π, a timestamping mechanism τ, and a timing mechanism time, the tuple (Π, τ, time) implements *timestamped atomic broadcast* if it satisfies the following properties with all but negligible probability:

Monotonicity. If honest process i has global timestamp $\tau(\mathit{VIEW}(i, t))$ at time t, then for every honest party j there exists a time t' such that $\tau(\mathit{VIEW}(j, t')) > \tau(\mathit{VIEW}(i, t))$.

Boundedness. At time t, there is at least one honest party i with local timestamp $\tau(i,t) \geq \tau(\mathit{VIEW}(i,t))$.

Completeness. If honest party i has global timestamp $\tau(\mathit{VIEW}(i,t))$ at time t, then i has *ab-delivered* every honestly submitted transaction before time $\tau(\mathit{VIEW}(i,t))$.

Awareness. Given a transaction T, there exists at least one honest party i such that $T \in \mathit{VIEW}(i,t)$ for some time $t \leq \mathit{time}(T)$.

Irreversibility. If honest party i has at time t global timestamp $\tau(\mathit{VIEW}(i,t))$, then for any *ab-delivered* transaction T after t, $\tau(T) \geq \tau(\mathit{VIEW}(i,t))$.

We stress that timestamping an atomic broadcast protocol does not alter the security guarantees of the underlying protocol. This will be clear from the constructions introduced by this paper, as the main consensus mechanism is not altered, and we will discuss this aspect in detail with each construction.

3 Protocols

3.1 General construction

We provide a general framework to timestamp an atomic broadcast protocol. Our frame can be divided in two distinct components: the computation of local timestamps and the derivation of the global timestamp from the reported local timestamps. The former is done independently of the atomic broadcast protocol Π considered, the latter needs to be adapted to the concrete protocol Π. In the following section, we describe the timestamping mechanism for the three main families of atomic broadcast protocols.

Local timestamps. Given an honest party i, its local timestamp, $\tau(i,t)$ is initialized to zero at the beginning of the protocol execution and updated to the maximum value τ, according to the conditions below.

1. Every transaction received before time $\tau - \Delta_u - 2\Delta_c$ has been delivered.
2. The timestamp is bounded by the current time, i.e., $\tau(i,t) \leq t$.

In plain terms, the local timestamp $\tau(i,t)$ takes the maximum value τ such that i has already *ab-delivered* any honestly submitted transaction before time $\tau(i,t)$, see Figure 1 for details.

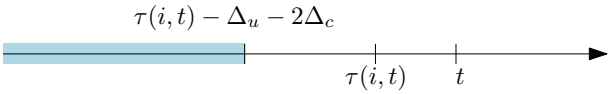

Figure 1. Example of the local timestamp for party i at time t. The local timestamp $\tau(i,t)$ guarantees that all transactions submitted before $\tau(i,t) - \Delta_u - 2\Delta_c$ (i.e., in the blue portion) are delivered.

Global timestamps. Given an honest party i, its global timestamp $\tau(\mathit{VIEW}(i,t))$ is computed deterministically from its view of the protocol $\mathit{VIEW}(i,t)$ at time t. More precisely, party i considers the set of blocks $\mathcal{B}$ in $\mathit{VIEW}(i,t)$ that it has already *ab-delivered* and applies some function that limits the Byzantine influence. Examples of these functions are the median for longest chain protocols, or quorum conditions for leader-based and DAG-based protocols.

3.2 A stable global timestamp for DAG-based consensus

In this section, we show how to construct stable global timestamps for the family of DAG-based protocols by timestamping DAG-Rider [29], as the original paper starting this line of work. The rest of the protocols share the same structure [29, 18, 44, 30, 24, 42, 14, 9, 8], and the timestamping mechanism generalizes straightforwardly.

DAG-Rider in a nutshell. DAG-Rider [29] is a permissionned round-based protocol where processes exchange messages using *reliable broadcast* [16]. Honest processes in round r wait until they reliably deliver at least $(n-f)$ blocks from round $r-1$, which they add to their new block as references. Honest processes conclude the round by reliably broadcasting the new block and moving to round $r+1$. In DAG-Rider, rounds are grouped in sets of 4 rounds, denoted as *waves*. When honest parties finalize the fourth round, the wave is completed and the attempt to deliver blocks is made. When the wave is completed, parties release a common coin that elects an *anchor block* from the first round of the wave. If this block is referenced at least $n-f$ blocks from the last round, the anchor is committed and all its past, i.e., blocks it refers to, is atomically delivered in a fixed deterministic order. We define the number of blocks from the last with $B \in \mathit{past}(B)$ as $\mathit{supp}(B)$. For a full description of DAG-Rider we refer to the original publication [29].

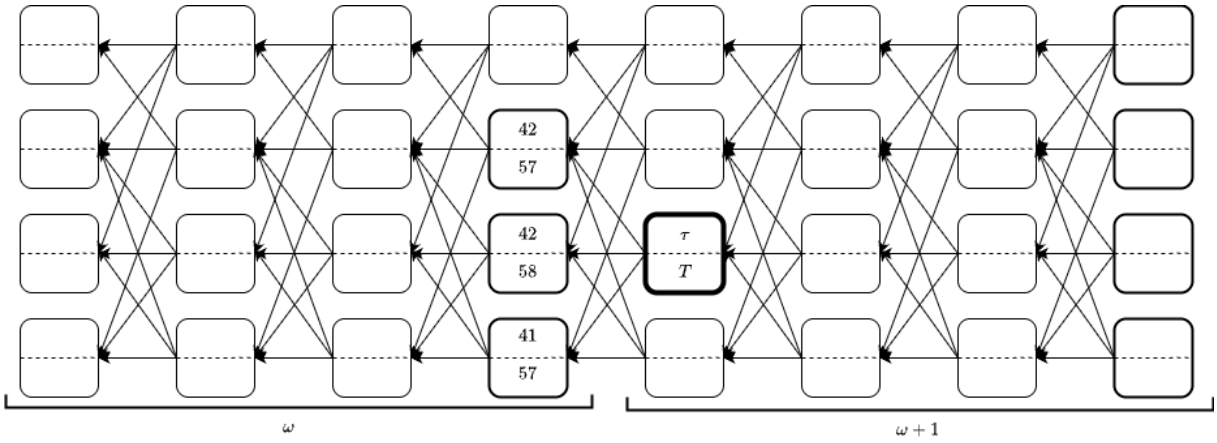

Figure 2. Example of the timestamp mechanism of DAG-Rider. This figure shows two consecutive waves from the view of an honest party i, every block contains a timestamp and a time, we omit them in irrelevant blocks for ease of reading. The leader of wave $\omega+1$ (marked in bold and the variables τ and T) define the global timestamp $\tau(\mathit{VIEW}(i,t))$ as it is the last committed anchor. Note that this block has enough support as indicated by the blocks in the last round of wave $\omega+1$. The timestamp and time of the block are computed to be the median of the reported timestamps (42) and time (57) of its parents, respectively. Note that the time is only used for transactions in the past of the block.

Timestamping DAG-Rider. An honest process i adds to every block it produces its local timestamp, according to the rules described in the general construction and the local time at the creation of the block. Given the view of an honest party i at time t, the global timestamp is computed based on the last committed block $\mathit{last}(i,t)$ in the view of honest i, according to the following simple set of rules. The global timestamp $\tau(\mathit{VIEW}(i,t))$ is

$$\max\Big\{\max_{t'<t}\big\{\tau(\mathit{VIEW}(i,t'))\big\}, \mathit{median}(\mathit{last}(i,t))\big\}\Big\}. \tag{1}$$

Note that views are monotone, i.e., $\mathit{VIEW}(i,t) \subseteq \mathit{VIEW}(i,t')$ for any two times $t' \geq t$. If a party creates a block with timestamp τ in round r, any block from a later round $r' > r$ must have a timestamp $\tau' \geq \tau$. Otherwise, the block is considered invalid. Figure 2 provides an example of timestamping DAG-Rider.

Timestamping transactions. Given an *ab-delivered* transaction T, its timestamp is computed to be the protocol timestamp of the latest anchor block $B \in \mathit{past}(\mathit{block}(T))$ with support $n-f$ of greater, i.e, $\max\{\mu \in \mathbb{N} : \exists B \in \mathit{VIEW}(i,t) : \tau(B) = \mu \wedge B \in \mathit{past}(\mathit{block}(T)) \wedge \mathit{supp}(B) \geq n-f\}$. This guarantees that for honest transactions the timestamp will be lower or equal to the time of submission. It also guarantees that once a party achieves $\tau(\mathit{VIEW}(i,t)) = C$, it will not deliver any transaction T with a timestamp $\tau(T) < C$. For consistency, when a new anchor block is committed and thus, the

global timestamp is possibly updated, the party first atomically delivers the transactions triggered by the commitment and then updates the timestamp. This prevents the idling transactions from possessing higher timestamps than required.

Timing transactions. Given an *ab-delivered* transaction T, its execution time is estimated as the median time of the parents of the next committed anchor block with $\mathit{block}(T)$ in its past through at least $n-f$ parties. This guarantees that at least one honest party has $\mathit{block}(T)$ in its local view before $\mathit{time}(T)$.

Lemma 1. *If an honest party commits an anchor block B in round r, then for every committed block B' by any honest party in round $r' > r$, then $B \in \mathit{past}(B')$.*

Proof. [29, Lemma 1]. □

Theorem 2. *DAG-Rider with the timestamp mechanism described above implements timestamped atomic broadcast.*

Proof. We structure the proof property by property.

Monotonicity. Consider an honest party with global timestamp $\tau(\mathit{VIEW}(i,t))$ at time t. The validity property of DAG-rider ([29], proposition 4) guarantees that at any point in time, party i eventually *ab-delivers* a block at time $T' \geq T$. Since honest parties received a finite amount of transactions before time T. Every honest party eventually creates a block with local timestamp greater than $\tau(\mathit{VIEW}(i,t))$. Honest party j eventually *ab-delivers* a block that it has produced after the slowest party has created such block, let's denote the time t'. When this happens, $\tau(\mathit{VIEW}(j,t')) > \tau(\mathit{VIEW}(i,t))$, as every honest party has local timestamp greater $\tau(\mathit{VIEW}(i,t))$ and the median of the ancestors of the anchor block is greater than $\tau(\mathit{VIEW}(i,t))$.

Boundedness. Consider a party i with the highest value of $\tau(\mathit{VIEW}(i,t))$ among the honest parties. The global timestamp according to i is computed as the median of a set blocks of at least $n-f$ parties. By definition of the median, there are at least $f+1$ blocks with a reported timestamp higher than $\tau(\mathit{VIEW}(i,t))$. At least one of those blocks B has been created by an honest party j, $\tau(\mathit{VIEW}(i,t)) \leq \tau(\mathit{VIEW}(i,t)) \leq \tau(B) \leq \tau(j,t)$.

Completeness. Assume that honest party has a global timestamp $\tau(\mathit{VIEW}(i,t))$. By definition of the global timestamp, party i has committed an anchor A such that the median timestamp of its parents is $\tau(\mathit{VIEW}(i,t))$. Thus, there exist $n-f$ parties that created a block in $\mathit{past}(A)$ reporting local timestamps greater or equal than $\tau(\mathit{VIEW}(i,t))$, thus at least one block B from honest party j. By definition of the local timestamp of j, party j has every transaction it has received before time in $\mathit{past}(B)$ $\tau(\mathit{VIEW}(i,t)) - \Delta$ as it has *ab-delivered* them. As an honest user sends its transactions to every party, thus i has *ab-delivered* every honestly submitted transaction before $\tau(\mathit{VIEW}(i,t))$.

Awareness. Given a transaction T, $\mathit{time}(T)$ is computed as the median time of the parents of the next committed block B that sees $\mathit{block}(T)$ through $2f+1$ processes. Thus, there exist $f+1$ parties, at least one honest, that reported a time lower or equal to $\mathit{time}(T)$ among the parents of B. Thus, there is an honest party i with $T \in \mathit{VIEW}(i,t)$ for some time $t < \mathit{time}(T)$.

Irreversibility. We recall that given a transaction T its timestamp is defined to be the timestamp of the last anchor block in its past with enough support. Consider an honest party at time t with global timestamp $\tau(\mathit{VIEW}(i,t))$ that *ab-delivers* a transaction T. Consider the anchor block B defining $\tau(\mathit{VIEW}(i,t))$, note that it satisfies $\mathit{supp}(B) \geq n-f$. Denote by B' the anchor block defining τT and recall that B' is the anchor from the highest round with a support of at least $n-f$. Lemma 1 implies $B' = B$ or $B \in \mathit{past}(B')$. Thus $\tau(T) \geq \tau(\mathit{VIEW}(i,t))$ and the irreversibility property holds.

□

3.3 A stable global timestamp for leader-based consensus

Here, we describe how to construct stable global timestamps for leader-based consensus protocols. We describe the global timestamps for PBFT [17] as it is the first practical leader-based atomic broadcast protocol [16], and we note that most of the notable follow-up work on practical leader-based consensus shares a similar overall structure to PBFT [34, 52].

PBFT in a nutshell. PBFT is a permissionless synchronous atomic broadcast protocol. We consider a set of n parties, up to $f < n/3$ may be corrupted by the adversary and arbitrarily diverge from the execution of the protocol. Parties communicate through a synchronous network with delay Δ, the adversary decides the delivery the messages within the Δ-time limit. One of the parties is selected to be the *leader* and create the blocks while the others take the secondary role of confirming blocks as *replicas*, the leader also plays the role of a replica. PBFT is divided in different subroutines:

Request. When a replica triggers *ab-broadcast* for a transaction T, it forwards a request message with the transaction to the leader. This phase occurs independently of the other phases

Pre-prepare. The leader is ready to begin an instance only after receiving $n - f$ replies from the current instance. Then, the leader sends a new block containing a set of requested transactions in a pre-prepare message to every replica.

Prepare. Upon receiving a pre-prepare message, the replicas verify the validity of the block. If the check succeeds, each replica sends a *prepare* message to every replica.

Commit. Upon receiving at least $n - f$ prepare messages, each replica sends a *commit* message to every replica. Once the replica has received $2f + 1$ commit messages, it *ab-delivers* the block B and sends a reply message to the leader indicating their readiness to move to the next round.

Leader-change. When a replica determines that the leader is corrupted it initiates a leader-change procedure. Once enough replicas have triggered this subroutine, a new leader is elected and the protocol continues from the last block that received enough reply messages.

Timestamping PBFT. Counterintuitively, the leader does not play any special role in determining the timestamp of a block. The replicas simply add their local timestamp as well as local times to the ready message they sent to the leader when concluding the round. The leader then selects $n - f$ of reported timestamps and adds them to the new block B. The timestamp of block B is then computed as the maximum between the median of the reported values and the timestamps of the already *ab-delivered* blocks. A replica that receives a block B with a timestamp $\tau(B)$ considers the block invalid if it has sent a transaction T to the leader before time $\tau(B) - \Delta_v$ that has not been added to B or another previously *ab-delivered* block. If enough replicas consider a block invalid, a change of leader is triggered. In particular, if a malicious leader does not include an honest transaction, every honest replica eventually halts, triggering a change of leader. This honest transactions from being censored.

Timestamping transactions. Given a transaction T contained in a block B, the timestamp of T is inherited from the timestamp of the block B, i.e., $\tau(T) = \tau(\mathit{block}(T))$. If block B is does not have a valid timestamp (i.e., if its maliciously generated), all the transactions $T \in B$ will also not have an assigned timestamp.

Timing transactions. The leader takes the median of all the local times of the $n - f$ replicas chosen to get the local time of a block B proposed by the leader. The time of a block $\mathit{time}(B)$ is computed as the maximum between the median of the reported values and the time of the already *ab-delivered* blocks. Given a transaction T, its time is inherited from the block that contains T, i.e., $\mathit{time}(T) = \mathit{time}(\mathit{block}(T)) + \delta_v$. Note that the leader cannot report an arbitrary time, as parties are aware of the time draft and the network delay.

Theorem 3. *PBFT with the timestamp mechanism described above implements timestamped atomic broadcast.*

Proof. We structure the proof property by property.

Monotonicity. Consider honest parties i and j such that i has a global timestamp $\tau(\mathit{VIEW}(i,t))$ at time t. Further, assume that the leader k is honest. The validity property of PBFT, guarantees that the leader k creates blocks that are *ab-delivered*. This block reports the median of the local timestamps of the $n-f$ replicas. Note that the local timestamp of every honest party monotonically increases over time. Using the argument above, party j eventually, say at time t', *ab-delivers* a block B with $\tau(B) > \tau(\mathit{VIEW}(i,t))$, thus $\tau(\mathit{VIEW}(j,t')) > \tau(\mathit{VIEW}(i,t))$.

Consider now the case that the leader k is corrupted. Since the timestamp is computed by the answers of the replicas, the only influence that a malicious leader has on the timestamps is on the creation of the blocks. In such a case, the honest replicas eventually initiate a change-of-leader procedure. This brings us back to the honest leader case above.

Boundedness. Consider the block B defining $\tau(\mathit{VIEW}(i,t))$, the timestamp of block B is computed as the median of $n-f$ local timestamps from the replicas, thus there must be one honest party i among the $n-f$ replicas that submitted a timestamp $\tau(i,t')$ larger or equal than the median for some $t' \leq t$. From the monotonicity property of local timestamps, $\tau(i,t') \leq \tau(i,t)$. Thus, boundedness holds for the honest party i, i.e., $\tau(i,t) > \tau(\mathit{VIEW}(i,t))$.

Completeness. Consider an honest party i with timestamp $\tau(\mathit{VIEW}(i,t))$. Party i adopts its global timestamp from an *ab-delivered* block B, i.e., $\tau(\mathit{VIEW}(i,t)) = \tau(B)$. The global timestamp $\tau(B)$ is computed by taking the median of the local timestamps of $n-f$ replicas, thus there is at least one honest party (say party j) included in the set of $n-f$ replicas that reported a local timestamp $\tau(j,t)$ such that $\tau(j,t) \geq \tau(B)$. This means that party j *ab-delivered* all honestly-submitted transactions before $\tau(j,t) - \Delta_u \geq \tau(B) - \Delta_u$. Since honest users send their transactions to all other users, all other honest parties will also have *ab-delivered* all honestly-submitted transactions before $\tau(B) - \Delta_u$.

Awareness. We recall that for a transaction T contained in some block B, $\tau(T)$ is given from a valid block B that contains T. This means that the leader must have received at least $n-f$ "ready" messages and local timestamps from the replicas. As the quorum is of size $n-f$ and $f < \frac{n}{3}$, there is at least one honest party (say party i) who submitted a local timestamp that is at upper bounded by the median of the $n-f$ local timestamps chosen by the leader. The exact same argument (since we assume no clock drift) shows that i's local time is no greater than the median of all $n-f$ other local times, which is exactly $\mathit{time}(T) = \mathit{time}(B)$. From the fact that local timestamps are defined to be upper bounded by local times, there is some time $t < \mathit{time}(T)$ where $T \in \mathit{VIEW}(i,t)$ for honest party i.

Irreversibility. Suppose at time t an honest party i has global timestamp $\tau(\mathit{VIEW}(i,t))$, defined by block B, and suppose there is some transaction T that is *ab-delivered* after time t. Then, since the global timestamp of i at time t is determined by the timestamp of the honest block B at the round corresponding to time t. The timestamp is computed as the maximum timestamp of all *ab-delivered* blocks; thus, every *ab-delivered* after B has a timestamp no smaller than B. Hence, the irreversibility property is fulfilled.

□

3.4 A stable global timestamp for longest-chain protocols

We now move to longest-chain protocols and show how to construct global timestamps for Nakamoto consensus [39].

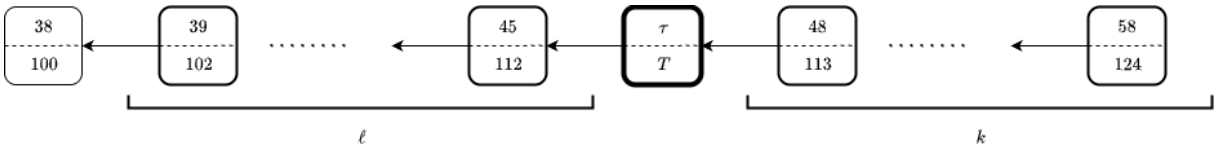

Figure 3. Example of timestamping of Nakamoto consensus. We consider the view of an honest party i. The block marked in bold with variables τ and T defines $\tau(\textit{view}(i,t))$ as it is the block in position $k+1$ from the tip of the chain. This value is computed as the median of the timestamp values of the ℓ immediate predecessors of the block, in this example, timestamps $39, ..., 45$. The time of a transaction contained in the block is estimated by the median of the reported time of the ℓ immediate successors.

Nakamoto in a nutshell. Nakamoto consensus [39] is a permissionless synchronous atomic broadcast protocol. Parties do not know the other parties running the protocol but have access to a gossiping functionality that guarantees that every honest party *hears* any *gossiped* messages after a maximum delay of Δ_v. Parties create blocks with references to previous blocks, creating a *directed acyclic graph* (DAG). When an honest party creates a block, it appends it at the end of the longest chain, i.e., adds a reference to the tip of the longest chain, elongating such chain. Two blocks are *forked* if they refer to the same block Garay, Kiayias, and Leonardos [23] have determined the security conditions of Nakamoto consensus. They coined the concept of *typical execution* a standard technique of the study of blockchain protocols [32, 7], we summarize the concept informally below, and refer to the original source for a precise definition [23]. For any period of time $[t, t+\lambda]$, the following properties are fulfilled.

- The number of honestly produced blocks $X([t,t+\lambda])$ during time $[t,t+\lambda]$ is bounded by its expected value, i.e, $(1-\varepsilon)\mathrm{E}[X([t,t+\lambda])] \leq X([t,t+\lambda]) \leq (1+\varepsilon)\mathrm{E}[X([t,t+\lambda])]$.
- The number of honestly produced blocks that are not forked by honest parties $Y([t,t+\lambda])$ is lower bounded: $(1-\varepsilon)\mathrm{E}[Y([t,t+\lambda])] \leq Y([t,t+\lambda])$.
- The number of malicious blocks $Z([t,t+\lambda])$ are upper-bounded, i.e., $Z([t,t+\lambda]) \leq \mathrm{E}[Z([t,t+\lambda])] + \varepsilon\mathrm{E}[X([t,t+\lambda])]$.
- The are no predictions on the hash functions.

Under this assumption, Garay et.al [23] have shown three key properties that the bitcoin protocol satisfies. Below we reformulate these properties using the notation of this paper.

Definition 3 (Common prefix). The common prefix property with parameter $k \in \mathbb{N}$ states that for any two honest parties p_i and p_j adopting chains $C_i \in \textit{VIEW}(i,t_i)$ and $C_j \in \textit{VIEW}(j,t_j)$ with $t_i \leq t_j$ respectively, then $C_i^{\lceil k} \preceq C_j$.

Definition 4 (Chain Quality). The chain quality property with parameter $\mu \in \mathbb{R}$ and $\ell \in \mathbb{N}$ states that for any honest party p_i with chain $C_i \in \textit{VIEW}(i,t)$, it holds that for any set of ℓ consecutive blocks in C_i, the ratio of honest blocks is at least ℓ.

Definition 5 (Chain Growth). The chain growth property with parameters $t_0 \in \mathbb{N}$ and $g \in \mathbb{R}$ states that for any honest party p_i holding chain $C_i \in \textit{VIEW}(i,t)$ and $C_i' \in \textit{VIEW}(i,t+t_0)$ at time $t+t_0$, then $|C_i' - C_i| \geq g \cdot t_0$.

Timestamping Nakamoto. Each party adds the creation time and local timestamp whenever they create a block. Parties compute the timestamp of a block as the highest median of any ℓ consecutive previous blocks (we make sure that $\mu \geq 1/2$ as it is a reasonable value). The global timestamp in the view of party i is the block timestamp of the $k+1$ latest block in its longest chain. The timestamp of the transaction is defined to be the timestamp of the block in which it is contained. Figure 3 depicts our timestamping mechanism on Nakamoto consensus.

Timing transactions. Given an *ab-delivered* transaction T included in block B, $\mathit{time}(T)$ is defined to be the median of the creation time of the ℓ blocks immediately after B. Note that honest parties may differ in the value of $\mathit{time}(T)$ until their local chains grow and the ℓ blocks become part of the k-common prefix and thus *ab-delivered*. However, regardless of the chain used to determine $\mathit{time}(T)$, the awareness probability holds.

Theorem 4. *Nakamoto consensus, with the timestamping mechanism described above, implements timestamped atomic broadcast protocol.*

Proof. We structure the proof property by property.

Monotonicity. Consider an honest party i with global timestamp $\tau(\mathit{VIEW}(i,t))$ at time t. The chain growth property states that given any length L, there exists a time t_0 such that for every honest party j the main chain in $\mathit{VIEW}(j,t')$ has a length greater than L at any time $t' \geq t_0$. Since honest parties receive a finite amount of transactions before time $\tau(\mathit{VIEW}(i,t))$, every honestly mined block has a timestamp greater than $\tau(\mathit{VIEW}(i,t))$. Eventually, party j *ab-delivers* a block B' such that all honest blocks in its ℓ most recent ancestors have a timestamp greater than $\tau(\mathit{VIEW}(i,t))$, let's denote this time t'. Finally, the chain quality property guarantees an honest majority of blocks among any ℓ consecutive ancestors, thus $\tau(\mathit{VIEW}(j,t')) > \tau(\mathit{VIEW}(j,t'))$ and the monotonicity property holds.

Boundedness. Consider the honest party j with the highest global timestamp, $\tau(\mathit{VIEW}(j,t))$, at time t among the honest parties. This timestamp is computed as the highest median of any ℓ consecutive ancestors of the last *ab-delivered* block B by j. The chain quality property guarantees that there exists an honest block B' such that $\tau(B') \geq \tau(\mathit{VIEW}(j,t))$. Let us denote the honest miner of B' by i, thus $\tau(i,t) \geq \tau(B') \geq \tau(\mathit{VIEW}(j,t)) \geq \tau(\mathit{VIEW}(i,t))$, proving boundedness.

Completeness. Consider an honest party i with global timestamp $\tau(\mathit{VIEW}(i,t))$ at time t and denote by B the block defining this timestamp. By definition $\tau(\mathit{VIEW}(i,t)) = \mathit{median}(B)$, together with the chain quality property, there exists an honest block B' among the ℓ consecutive ancestors ancestors of B defining the timestamp such that $\tau(B') \geq \tau(\mathit{VIEW}(i,t))$, note that B' has already been *ab-delivered*. The creator of B' is an honest party j with timestamp $\tau(j,t) = \tau(B') \geq \tau(\mathit{VIEW}(i,t))$, i.e., at the time of mining of B, j has *ab-delivered* every transaction received before time $\tau(\mathit{VIEW}(i,t))$. Finally, note that the set of transactions delivered by i contains the set of transactions delivered by j at the time of creation of B', since B' is an ancestor of B. Thus, the completeness property is fulfilled.

Awareness. Consider an honest party i and an *ab-delivered* transaction T that is part of block B, $\mathit{time}(T)$ is defined to be the median time of the ℓ immediate descendants of B. The chain quality property guarantees that there is at least one honest party that created a block B' descendant of B such that $\mathit{time}(B) \leq \mathit{time}(T)$ according to $\mathit{VIEW}(i,t)$, the local state of party i for any. The chain growth property and common prefix properties guarantee that these ℓ descendants are the same blocks in the view of every honest party. Thus, the awareness property holds.

Irreversibility. Consider an honest party i with timestamp $\tau(\mathit{VIEW}(i,t))$ at time t. Further assume that i has *ab-delivered* a transaction T at time $t' \geq t$, we will show that $\tau(T) \geq \tau(\mathit{VIEW}(i,t))$. By definition, $\tau(T)$ is the highest median among ℓ consecutive blocks in the past of B, whereas $\tau(\mathit{VIEW}(i,t))$ is the highest median among ℓ consecutive blocks in the past of the last delivered block before time t. Using the common prefix property, $\tau(T)$ is the maximum over a set of blocks that is a superset of the set determining $\tau(\mathit{VIEW}(i,t))$, thus $\tau(T) \geq \tau(\mathit{VIEW}(i,t))$ and the irreversibility property is fulfilled.

□

4 Layer 2 Solutions for Timestamped Consensus Protocols

We show that our timestamping protocol enables meaningful layer 2 protocols on top of layer 1 protocols, implementing our timestamping functionality. In Section 4.1, we introduce general abstractions for layer 1–layer 2 interactions. In Section 4.2, we define security guarantees for these interactions and prove that layer 2 protocols built on timestamped consensus satisfying the abstractions of Section 4.1 meet them.

4.1 Abstract Layer 2 Functionalities

Let S be the set of users in the blockchain protocol (both clients and consensus participants). We abstract layer 1–layer 2 interactions with the following functionalities:

- $\mathsf{Open}(S' \subset S, \mathcal{I}, \mathcal{O})$: takes a subset of users S' and inputs $\mathcal{I}$, processes them, and outputs $\mathcal{O}$. The storage location of outputs (on-chain or off-chain) is left unspecified.
- $\mathsf{Close}(S' \subset S, \mathcal{I}, \mathcal{O} = \{\mathit{state}, t, \dots\})$: similar to Open but $\mathcal{O}$ must include: (i) a state object *state* recording the closing state of the Layer 2 protocol, and (ii) a protocol-defined timeout t giving an upper bound for contesting the closing state. The timeout cannot be chosen by the caller. The set of involved users may differ from Open (e.g., unilateral channel closure).
- $\mathsf{Contest}(\mathcal{I} = \{t, \mathit{state}, \mathit{state}_c, c\}, \mathcal{O} = \{\mathit{state}_f, \pi, \dots\})$: allows a party to contest a closing state *state* with an alternative state_c using contest input c, which yields a proof π. If c is valid and submitted before t, the final state is state_c; otherwise it remains *state*. The timeout t must match that of Close. This functionality is always invoked after Close.

We formalize these in Algorithm 1, and conjecture that interactions between any layer 2 protocol and its blockchain can be abstracted accordingly. We illustrate this with payment channels, optimistic rollups, and ZK rollups.

Example 1 (Payment channels). For users $s_1, s_2 \in S$ opening a payment channel, Open takes deposits b_1, b_2 and outputs a blockchain transaction recording the initial balances and the new channel object. Close may be invoked unilaterally or jointly, outputting a transaction with the final balances (the *state* object). In unilateral closure, the counterpart has until time t to submit a fresher signed state state_c, serving as the contest input c, thereby overriding the closing state.

Example 2 (Optimistic rollups). Open takes a set of transactions and outputs a compressed transaction blob. Close publishes this blob as a batch on-chain, forming the closing state *state*. In Contest, a party has until t to submit an assertion c challenging the rollup state from Close.

Example 3 (ZK rollups). Open takes a set of transactions and outputs a compressed blob with a ZK proof of validity. Close verifies the proof and executes the blob on-chain. Since validity is guaranteed by the proof, Contest is trivial (identity with $t = 0$ and $c = \emptyset$).

Algorithm 1 is an abstraction of the *core functionalities* in layer 2–layer 1 interactions. Some layer 2 protocols may deviate from the exact procedures—for example, verification of contested states and times may occur in Close instead of Contest. Our focus, however, is on protocols whose specific algorithms align with the general functionalities captured by Algorithm 1.

4.2 Ideal guarantees

Let Π be a blockchain consensus protocol implementing timestamped atomic broadcast with timestamping τ and timing *time*. We show that if times can be processed and output by some function, then Π can support a layer 2 protocol interacting with it as in Algorithm 1.

Theorem 5. *If we have a procedure that can process and output times, any layer 1 consensus protocol* Π *can be extended with a layer 2 protocol that interacts with* Π *following the specifications of Algorithm 1.*

Algorithm 1 An abstraction of the interaction between Layer 1 and Layer 2 protocols

1: **Inputs** set of users S, open input list $\mathcal{I}_o$, close input list $\mathcal{I}_c$

2: **procedure** Open($S' \subset S, \mathcal{I}_o$) **do**
3: output *process*($S', \mathcal{I}_o$)

4: **procedure** Close($S' \subset S, \mathcal{I}_c$) **do**
5: *state*, *aux*, t = *process*($S', \mathcal{I}_c$)
6: output (*state*, *aux*, t)

7: **procedure** Contest($S' \subset S, t, state, state_c, c$) **do**
8: $t_{curr} = getSubmissionTime()$
9: $state_f, \pi$ = *process*($S', state, state_c, c$)
10: **if** (verify(π) $\wedge$ $t_{curr} < t$) **then**
11: output ($state_f, \pi$)
12: output (*state*, π)

Proof. Follows from the inputs and outputs as defined by the process procedures in lines 5 and 9 in Algorithm 1. □

We stress that Theorem 5 is non-trivial. It shows that if the timestamping mechanism guarantees an upper bound on when all honest parties receive submitted transactions (cf. lines 5, 9 of Algorithm 1), then layer 2 protocols can be built on DAG-based layer 1 consensus. Moreover, in Definition 6 and theorem 6 we prove not only this upper-bound guarantee for consensus with our timestamping mechanism, but also resistance to bribing attacks—something no existing DAG-based protocol achieves.

Let Π_{L_2} denote a layer 2 protocol built on top of Π, implementing both the functionalities of Algorithm 1 and those of Π. We next introduce the security guarantees that ideal layer 1–layer 2 interactions should satisfy.

Definition 6 ((t_1, t_2, t_3)-ideal layer 2 timing security). For times $t_1 < t_2 \leq t_3$, as long as the number of Byzantine parties f is such that the security of the underlying consensus protocol Π is satisfied, and the contested state $state_c$ is submitted between t_1 and t_2, then *time*($state_c$) $< t_3$.

We observe that Definition 6 ensures not only that honest users can contest fraudulent closing states if submitted in time, but also that their contested state will appear on-chain by some time t_3. We now state and prove our main guarantee: any Π_{L_2} built on timestamped consensus is secure as in Definition 6.

Theorem 6. Π_{L_2} *satisfies* (t_1, t_2, t_3)*-ideal layer 2 timing security.*

Proof. Since we are only interested in security of layer 2 protocols for honest parties, we can assume that wlog an honest party submitted the contested state $state_c$. We also limit our analysis to closing states which have seen by all honest parties and can be contested. Note that due to the assumption on the security of Π, states that are not seen by all honest parties either will never live on the blockchain (and thus there is no point contesting them), or will eventually be delivered and seen by all honest parties. Therefore, we can reduce the analysis in this case to the case of honestly submitted states.

From the completeness property of Π, we first define t_1 as the time in which all honest parties delivered the closing state *state*, i.e., $t_1 = \tau(VIEW(i,t)) \mid state \in VIEW(i,t)$ for all i and for some time $t \geq t_1$. This means that at time t, all honest parties will see $state$ on the blockchain, i.e., $state \in VIEW(i,t)\ \forall i$. From the monotonicity and completeness property of Π, an honestly submitted contested state $state_c$ submitted by time $t_2 = t + \delta$ for some $\delta > 0$ will be delivered by all honest processes i by time $\tau(VIEW(i,t_3))$ for some $t_3 \geq t_2$. Thus, we can guarantee that any honestly submitted contested state between time t_1 and t_2 will be delivered by all honest parties by time t_3. □

We end this section with three remarks.

Remark 1 (Bribery resistance). Theorem 6 guarantees bribery resistance as long as the underlying consensus protocol Π remains secure, i.e., the number of Byzantine parties does not exceed its threshold. Thus, layer 2 protocols on timestamped consensus are resistant to bribery and feather-forking attacks [12, 37], where adversaries bribe others to exclude transactions (e.g., fraudulent closures blocking honest contests). Since timestamping enforces an upper bound on delivery time for honest transactions, such attacks fail whenever Byzantine participation is below the security threshold.

Remark 2 (Security under network congestion). Beyond bribery resistance, Theorem 6 also guarantees security during network congestion, by relying on the timestamped atomic broadcast properties of Definition 2. Even under congestion, contested states are still delivered and executed on-chain, with only the exact timing parameters in Theorem 6 (see Remark 3) potentially changing.

Remark 3 (On the choice of t_2 and t_3). Theorem 6 and our protocol guarantee only the existence of times t_2 and t_3, ensured by timestamped atomic broadcast. Estimating their concrete values for different consensus protocols under varying congestion is an important but orthogonal problem, left for future work.

5 Conclusion

In this work, we present a novel timestamping mechanism for consensus protocols and formally detail how to implement our timestamping mechanism for DAG-based, leader-based, and longest chain consensus protocols. We prove that the extending these consensus protocols with our timestamping mechanism not only preserves the existing security properties of the protocols, but also equips these protocols with a notion of submission time and guarantees that honest transactions submitted by some time will be delivered by some other time. We also show how to construct secure, layer 2 solutions on top of our timestamped protocols, and prove that timestamping allows our layer 2 solutions to remain secure under bribery attacks and network congestion.

References

[1] Cartesi. https://cartesi.io/cartesi_lightpaper_english.pdf (2020)

[2] Fuel network. https://fuel.sh/ (2020)

[3] Liquid. https://liquid.net/ (2020)

[4] Offchain lab's arbitrium. https://arbitrum.io/ (2020)

[5] Optimism. https://www.optimism.io/ (2020)

[6] Polygon's pos. https://polygon.technology/solutions/polygon-pos (2020)

[7] Amores-Sesar, I., Cachin, C., Parker, A.: Generalizing weighted trees: a bridge from bitcoin to GHOST. In: AFT. pp. 156–169. ACM (2021)

[8] Amores-Sesar, I., Grøndal, V., Holmgård, A., Ottendal, M.: Dag it off: Latency prefers no common coins. CoRR **abs/2508.14716** (2025)

[9] Arun, B., Li, Z., Suri-Payer, F., Das, S., Spiegelman, A.: Shoal++: High throughput DAG BFT can be fast and robust! In: NSDI. pp. 813–826. USENIX Association (2025)

[10] Aumayr, L., Avarikioti, Z., Maffei, M., Mazumdar, S.: Securing lightning channels against rational miners. In: CCS. pp. 393–407. ACM (2024)

[11] Avarikioti, G., Kokoris-Kogias, E., Wattenhofer, R.: Brick: Asynchronous state channels. CoRR **abs/1905.11360** (2019)

[12] Avarikioti, Z., Kedzior, P., Lizurej, T., Michalak, T.: Bribe & fork: Cheap PCN bribing attacks via forking threat. In: AFT. LIPIcs, vol. 316, pp. 11:1–11:22. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2024)

[13] Avarikioti, Z., Litos, O.S.T.: Suborn channels: Incentives against timelock bribes. In: Financial Cryptography. Lecture Notes in Computer Science, vol. 13411, pp. 488–511. Springer (2022)

[14] Babel, K., Chursin, A., Danezis, G., Kichidis, A., Kokoris-Kogias, L., Koshy, A., Sonnino, A., Tian, M.: Mysticeti: Reaching the latency limits with uncertified dags. In: NDSS. The Internet Society (2025)

[15] Bentov, I., Ji, Y., Zhang, F., Breidenbach, L., Daian, P., Juels, A.: Tesseract: Real-time cryptocurrency exchange using trusted hardware. In: CCS. pp. 1521–1538. ACM (2019)

[16] Cachin, C., Guerraoui, R., Rodrigues, L.E.T.: Introduction to Reliable and Secure Distributed Programming (2. ed.). Springer (2011)

[17] Castro, M., Liskov, B.: Practical byzantine fault tolerance. In: OSDI. pp. 173–186. USENIX Association (1999)

[18] Danezis, G., Kokoris-Kogias, L., Sonnino, A., Spiegelman, A.: Narwhal and tusk: a dag-based mempool and efficient BFT consensus. In: EuroSys. pp. 34–50. ACM (2022)

[19] Decker, C., Russell, R.: eltoo : A simple layer 2 protocol for bitcoin (2018), https://api.semanticscholar.org/CorpusID:49253813

[20] Decker, C., Wattenhofer, R.: A fast and scalable payment network with bitcoin duplex micropayment channels. In: Pelc, A., Schwarzmann, A.A. (eds.) Stabilization, Safety, and Security of Distributed Systems. pp. 3–18. Springer International Publishing, Cham (2015)

[21] Foundation, E.: Ethereum whitepaper, https://ethereum.org/en/whitepaper/

[22] Gangwal, A., Gangavalli, H.R., Thirupathi, A.: A survey of layer-two blockchain protocols. Journal of Network and Computer Applications **209**, 103539 (2023), https://www.sciencedirect.com/science/article/pii/S1084804522001801

[23] Garay, J.A., Kiayias, A., Leonardos, N.: The bitcoin backbone protocol: Analysis and applications. In: EUROCRYPT (2). Lecture Notes in Computer Science, vol. 9057, pp. 281–310. Springer (2015)

[24] Giridharan, N., Suri-Payer, F., Abraham, I., Alvisi, L., Crooks, N.: Autobahn: Seamless high speed BFT. In: SOSP. pp. 1–23. ACM (2024)

[25] Guerraoui, R., Kuznetsov, P., Monti, M., Pavlovic, M., Seredinschi, D.: The consensus number of a cryptocurrency. In: PODC. pp. 307–316. ACM (2019)

[26] Herlihy, M.: Wait-free synchronization. ACM Trans. Program. Lang. Syst. **13**(1), 124–149 (Jan 1991)

[27] Jaynti Kanani, Sandeep Nailwal, A.A.: Matic whitepaper. https://github.com/maticnetwork/whitepaper (2021)

[28] Kalodner, H.A., Goldfeder, S., Chen, X., Weinberg, S.M., Felten, E.W.: Arbitrum: Scalable, private smart contracts. In: USENIX Security Symposium. pp. 1353–1370. USENIX Association (2018)

[29] Keidar, I., Kokoris-Kogias, E., Naor, O., Spiegelman, A.: All you need is DAG. In: PODC. pp. 165–175. ACM (2021)

[30] Keidar, I., Naor, O., Poupko, O., Shapiro, E.: Cordial miners: Fast and efficient consensus for every eventuality. In: DISC. LIPIcs, vol. 281, pp. 26:1–26:22. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2023)

[31] Khalil, R.: NOCUST-A Non-Custodial 2nd-Layer Blockchain Payment Hub. Ph.D. thesis, Swiss Federal Institute of Technology, Zurich (2018)

[32] Kiayias, A., Panagiotakos, G.: On trees, chains and fast transactions in the blockchain. In: LATINCRYPT. Lecture Notes in Computer Science, vol. 11368, pp. 327–351. Springer (2017)

[33] Labs, M.: zksync: Bringing trustless, scalable payments to ethereum. https://zksync.io/ (2015)

[34] Lamport, L.: The part-time parliament. ACM Trans. Comput. Syst. **16**(2), 133–169 (1998)

[35] Magnani, A., Calderoni, L., Palmieri, P.: Feather forking as a positive force: incentivising green energy production in a blockchain-based smart grid. In: CRYBLOCK@MobiSys. pp. 99–104. ACM (2018)

[36] Marcus, Y., Heilman, E., Goldberg, S.: Low-resource eclipse attacks on ethereum's peer-to-peer network. IACR Cryptol. ePrint Arch. p. 236 (2018)

[37] Miller, A.: Feather-forks: enforcing a blacklist with sub-50

[38] Miller, A., Bentov, I., Bakshi, S., Kumaresan, R., McCorry, P.: Sprites and state channels: Payment networks that go faster than lightning. In: Financial Cryptography. Lecture Notes in Computer Science, vol. 11598, pp. 508–526. Springer (2019)

[39] Nakamoto, S.: Bitcoin: A peer-to-peer electronic cash system. Whitepaper (2009), http://bitcoin.org/bitcoin.pdf

[40] Poon, J., Dryja, T.: The bitcoin lightning network: Scalable off-chain instant payments. https://lightning.network/lightning-network-paper.pdf (2015)

[41] Shalini, S., .H, S.: A survey on various attacks in bitcoin and cryptocurrency. pp. 0220–0224 (04 2019)

[42] Shrestha, N., Kate, A., Nayak, K.: Sailfish: Towards improving latency of dag-based BFT. IACR Cryptol. ePrint Arch. p. 472 (2024)

[43] Shrestha, N., Shrothrium, R., Kate, A., Nayak, K.: Sailfish: Towards improving the latency of dag-based BFT. In: SP. pp. 1928–1946. IEEE (2025)

[44] Spiegelman, A., Giridharan, N., Sonnino, A., Kokoris-Kogias, L.: Bullshark: DAG BFT protocols made practical. In: CCS. pp. 2705–2718. ACM (2022)

[45] Spilman, J.: Anti dos for tx replacement. https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2013-April/002433.html (2013)

[46] Team, T.M.: Fast path and consensus, https://move-book.com/object/fast-path-and-consensus.html

[47] Teutsch, J., Reitwießner, C.: A scalable verification solution for blockchains. CoRR **abs/1908.04756** (2019)

[48] Tzinas, A., Sridhar, S., Zindros, D.: On-chain timestamps are accurate. In: Financial Cryptography and Data Security: 28th International Conference, FC 2024, Willemstad, Curaçao, March 4–8, 2024, Revised Selected Papers, Part I. p. 110–127. Springer-Verlag, Berlin, Heidelberg (2025)

[49] Wahrstätter, A., Ernstberger, J., Yaish, A., Zhou, L., Qin, K., Tsuchiya, T., Steinhorst, S., Svetinovic, D., Christin, N., Barczentewicz, M., Gervais, A.: Blockchain censorship. In: WWW. pp. 1632–1643. ACM (2024)

[50] Wüst, K., Matetic, S., Schneider, M., Miers, I., Kostiainen, K., Capkun, S.: Zlite: Lightweight clients for shielded zcash transactions using trusted execution. In: Financial Cryptography. Lecture Notes in Computer Science, vol. 11598, pp. 179–198. Springer (2019)

[51] Yeo, M., Zhang, H.: Rational censorship attack: Breaking blockchain with a blackboard. CoRR **abs/2507.01453** (2025)

[52] Yin, M., Malkhi, D., Reiter, M.K., Golan-Gueta, G., Abraham, I.: Hotstuff: BFT consensus with linearity and responsiveness. In: PODC. pp. 347–356. ACM (2019)